
CSC 123 – Programming and Problem Solving II

Lab 05 – CheckingAccount (Validation Review + processCheck)

Total Points: 10

Purpose

This lab builds directly on Lab 04.

In Lab 04, you implemented the `CheckingAccount` class with:

- Constructors
- Getters and setters
- Strict input validation

In Lab 05, you must:

1. Verify that all Lab 04 validation rules still work correctly.
2. Add and correctly implement the `processCheck(double amount)` method.
3. Demonstrate proper overdraft and fee behavior.
4. Implement and demonstrate transaction support methods:
 - `depositMoney(double amount)`
 - `withdrawMoney(double amount)`

This lab serves as both an extension and a validation audit of your Lab 04 work.

Lab 04 Validation Requirements (Must Still Work)

Your `CheckingAccount` class must enforce:

Account Name

- Must be a `String`
- Must contain at least two words
- First and last name required

Account Balance

- Must be a dollar value
- No negative balances allowed at creation or via setter
- No fractional cents allowed
 - Invalid: `100.001`
 - Invalid: `-5.00`

Account Number

- Must be a `String`
 - Must contain only digits
 - Must be exactly 8 digits long
-

Required Transaction Methods

You must include and correctly implement:

```
public void depositMoney(double amount)
public void withdrawMoney(double amount)
```

depositMoney(double amount)

- Must reject negative amounts.
- Must reject fractional cents.
- Must add the amount to the balance if valid.

withdrawMoney(double amount)

- Must reject negative or zero amounts.
- Must reject fractional cents.
- Must subtract the amount from the balance.
- Overdrafts NOT are allowed.

Validation logic must be consistent with Lab 04 balance rules.

NEW Requirement – processCheck Method

Add:

```
public void processCheck(double amount)
```

Behavior Specification

- Subtract the check amount from the balance.
- Overdrafts are allowed.
- If the balance is **less than \$1000.00 BEFORE processing the check**, charge a **\$0.15 fee**.
- The fee is applied **per check**.
- Fee decision must be based on the balance before subtraction.

Example Logic Behavior

Starting Balance Check Fee Applied? Result

1000.00	1.00	No	999.00
999.99	1.00	Yes	998.84
0.00	20.00	Yes	-20.15

Correct ordering matters.

File Requirements

- File name must be:

`CheckingAccount.java`

- Only submit this file.
- It must compile.
- No IDE folders.
- No zip files unless instructed.

Required Header

Your file must begin with:

```
/*****
 * Name: First Last
 * Course: CSC 123
 * Lab: 05
 * Date:
 * File: CheckingAccount.java
 *
 * Description:
 * CheckingAccount class with validated input and processCheck method.
 *****/
```

Missing header → formatting points deducted.

Output Requirements

You must demonstrate:

- Valid constructor usage
- Invalid constructor inputs rejected
- Setter validation still functioning
- `depositMoney` functioning correctly
- `withdrawMoney` functioning correctly
- `processCheck` correctly applying fees
- Overdraft behavior functioning

Output must clearly show:

- Balance before
- Check amount
- Fee (if applicable)
- Balance after

Readable formatting required.

Compilation Policy

If your file does not compile:

Maximum score = 5 points

No exceptions.

UML

	CheckingAccount

	- accountNumber : String
	- accountName : String
	- accountBalance : double

	+ CheckingAccount(accountNumber:String,
	accountHolder:String,
	balance:double)
	+ CheckingAccount()

	+ getAccountBalance() : double
	+ setAccountBalance(amount:double) : void
	+ setAccountName(name:String) : void
	+ getAccountName() : String
	+ setAccountNumber(number:String) : void
	+ getAccountNumber() : String
	+ depositMoney(amount:double) : void
	+ withdrawMoney(amount:double) : void
	+ processCheck(amount:double) : void
	+ toString() : String

	- isValidAmount(amount:double) : boolean

Grading Rubric (10 Points)

Category	Points
Output correctness (validation + transactions + processCheck)	5
Comments and formatting	3
Structure, naming, compliance	2

If it does not compile → cap at 5 points.
